





SESIÓN 07: HERENCIA Y POLIMORFISMO

Docente del Curso



Reflexión

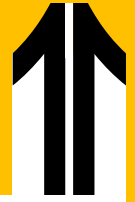


Puedes ver el video:

https://www.youtube.com/watch?v=yLj4WvKA8qI&ab_channel=TutoProgra



Al terminar la sesión el estudiante implementa herencia y polimorfismo aplicando la programación orientada a objetos y el uso del lenguaje de programación de Java en ejercicios propuestos por el docente.



¿Qué es la Herencia y por qué es importante?

¿Qué es la Herencia?

La herencia es un mecanismo fundamental en la programación orientada a objetos que permite crear nuevas clases (subclases o clases hijas) basadas en clases existentes (superclases o clases padres). **La subclase hereda atributos y métodos de la superclase, promoviendo la reutilización de código y estableciendo una relación "es-un" entre las clases.**

¿Por qué es Importante?

La herencia facilita la creación de jerarquías de clases bien estructuradas, modelando relaciones del mundo real de manera intuitiva. Reduce la duplicación de código, mejora la mantenibilidad y permite la extensión del comportamiento de las clases existentes sin modificarlas directamente, cumpliendo con el principio de abierto/cerrado.

RELACIÓN DE HERENCIA SIMPLE EN JAVA



La relación de herencia simple en Java implica que una clase puede heredar de otra única clase. Esto significa que una subclase solo puede heredar atributos y métodos de una clase padre

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

Ejemplos



```
class Shape {  
    void draw() {  
        System.out.println("Drawing a shape");  
    }  
}
```

```
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing a circle");  
    }  
}
```

```
class Vehicle {  
    void drive() {  
        System.out.println("Vehicle is being driven");  
    }  
}  
  
class Car extends Vehicle {  
    void park() {  
        System.out.println("Car is parking");  
    }  
}
```

Ejemplo de Código Java con UML



```
// Clase padre
class Vehiculo {
    String marca;
    int año;

    void mostrarInformacion() {
        System.out.println("Marca: " + marca + ", Año: " + año);
    }
}

// Clase hija
class Coche extends Vehiculo {
    int puertas;

    void mostrarInformacionCoche() {
        mostrarInformacion();
        System.out.println("Puertas: " + puertas);
    }
}
```



```
-----
|      Vehiculo      |
|-----|
| - marca: String    |
| - año: int         |
|-----|
| + mostrarInformacion()|
|-----|
|
| extends
|
|-----|
|      Coche        |
|-----|
| - puertas: int     |
|-----|
| + mostrarInformacionCoche()|
|-----|
```


¿Qué es el Polimorfismo? Una Visión General



Adaptabilidad

El polimorfismo permite que que un objeto tome muchas muchas formas.

Capacidad de una clase de de parecer y actuar como otra.



Flexibilidad

Facilita la escritura de código genérico y reutilizable, que puede operar sobre objetos de diferentes clases de manera uniforme.



Abstracción

Oculto la complejidad subyacente y presenta una una interfaz simplificada para para interactuar con los objetos.

Permite tratar objetos de diferentes clases de manera uniforme a través de una interfaz común. interfaz común.

No solo simplifica el código, sino que también promueve la extensibilidad y la mantenibilidad a mantenibilidad a largo plazo. Piénsenlo como la capacidad de un camaleón para adaptarse a su adaptarse a su entorno.

Tipos de Polimorfismo: Estático vs. Dinámico



Polimorfismo Estático (Sobrecarga)

También conocido como "early binding" o "compile-time polymorphism".

Se resuelve en tiempo de compilación. Implica tener múltiples métodos con el mismo nombre pero diferentes parámetros dentro de la misma clase.

Polimorfismo Dinámico (Sobreescritura)

También conocido como "late binding" o "runtime polymorphism".

Se resuelve en tiempo de ejecución. Se logra mediante la herencia y la sobreescripción de métodos en las subclases.

El polimorfismo estático se logra mediante la sobrecarga de métodos, donde la decisión de qué método ejecutar se toma en tiempo de compilación.

El polimorfismo dinámico, por otro lado, se basa en la sobreescripción de métodos en las subclases, las subclases, permitiendo que la decisión se tome en tiempo de ejecución, lo que añade flexibilidad al código.

Ejemplos:



```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator calculator = new Calculator();  
        System.out.println(calculator.add(1, 2));  
        System.out.println(calculator.add(1.5, 2.5));  
    }  
}
```

```
// Clase base abstracta
abstract class Animal {
    abstract void hacerSonido();
}

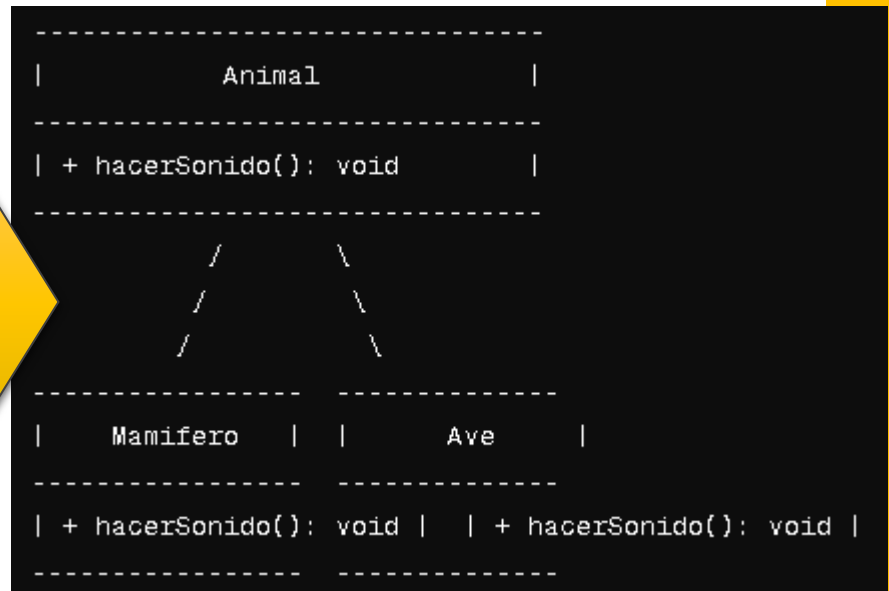
// Subclases
class Mamifero extends Animal {
    // Constructor, métodos y demás...
    void hacerSonido() {
        System.out.println("Sonido de mamífero");
    }
}

class Ave extends Animal {
    // Constructor, métodos y demás...
    void hacerSonido() {
        System.out.println("Sonido de ave");
    }
}

// Clase principal
public class Main {
    public static void main(String[] args) {
        Animal[] animales = new Animal[2];
        animales[0] = new Mamifero();
        animales[1] = new Ave();

        for (Animal animal : animales) {
            animal.hacerSonido();
        }
    }
}
```

Ejemplo con código Java y UML



Otro Ejemplo

1

```
// Clase base abstracta
abstract class Figura {
    abstract double area();
}

// Subclases
class Circulo extends Figura {
    double radio;
    // Constructor, métodos y demás...
    double area() {
        return Math.PI * radio * radio;
    }
}

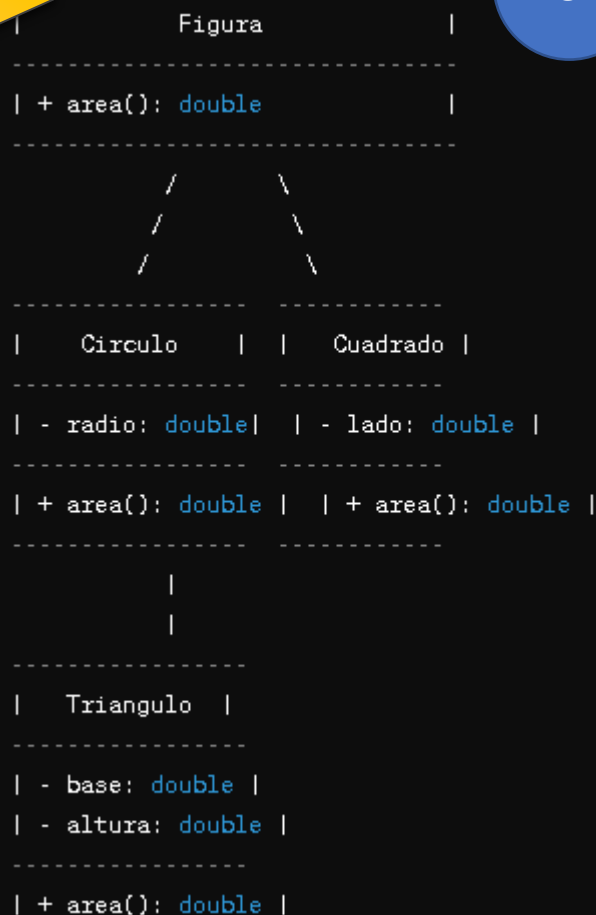
class Cuadrado extends Figura {
    double lado;
    // Constructor, métodos y demás...
    double area() {
        return lado * lado;
    }
}
```

```
class Triangulo extends Figura {
    double base;
    double altura;
    // Constructor, métodos y demás...
    double area() {
        return (base * altura) / 2;
    }
}
```

```
// Clase principal
public class Main {
    public static void main(String[] args) {
        Figura[] figuras = new Figura[3];
        figuras[0] = new Circulo();
        figuras[1] = new Cuadrado();
        figuras[2] = new Triangulo();
    }
}
```

2

3



CASTING EN JAVA



El casting en Java se refiere a la conversión de un tipo de dato a otro.

Se puede realizar tanto para tipos primitivos como para objetos.

```
double x = 1.23;
int y = (int) x;
System.out.println(y); // Output: 1
```

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }

    void playFetch() {
        System.out.println("Dog plays fetch");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog();
        Dog dog = (Dog) animal;
        dog.makeSound(); // Output: Dog barks
        dog.playFetch(); // Output: Dog plays fetch
    }
}
```

Ejemplos:

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }

    void playFetch() {
        System.out.println("Dog plays fetch");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal[] animals = new Animal[2];
        animals[0] = new Dog();
        animals[1] = new Animal();

        for (Animal animal : animals) {
            if (animal instanceof Dog) {
                Dog dog = (Dog) animal;
                dog.makeSound(); // Output: Dog barks
                dog.playFetch(); // Compiler error: playFetch()
            }
        }
    }
}
```



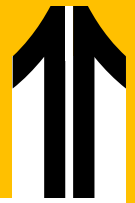
Otro Ejemplo



```
public class Main {  
    public static void main(String[] args) {  
        double decimal = 5.5;  
        int entero = (int) decimal; // Casting de double a int  
        System.out.println(entero); // Imprime 5  
    }  
}
```

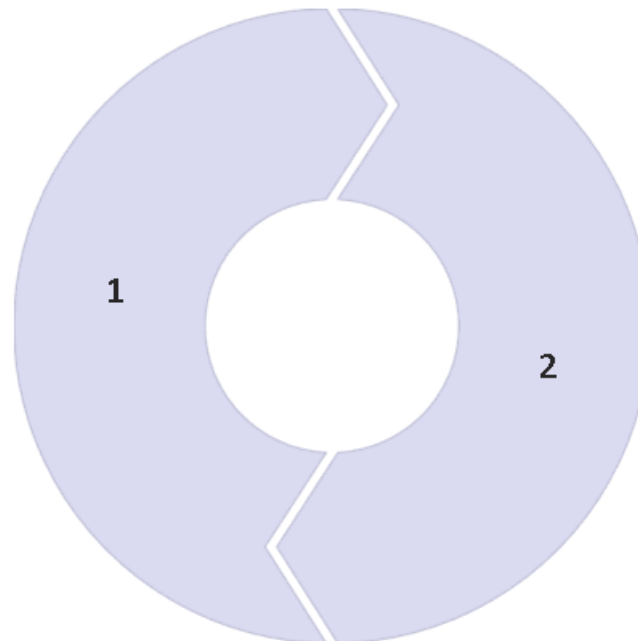
```
class Animal {  
    public void sonido() {  
        System.out.println("Hace algún sonido");  
    }  
}  
  
class Perro extends Animal {  
    public void sonido() {  
        System.out.println("Guau");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal animal = new Perro();  
        Perro perro = (Perro) animal; // Casting de Animal a Perro  
        perro.sonido(); // Imprime "Guau"  
    }  
}
```


Casting en Java: Upcasting y Downcasting



Upcasting

Conversión de una subclase a una superclase. Implícito y seguro.



Downcasting

Conversión de una superclase a una subclase. Explícito y requiere precaución.

Riesgos y Beneficios del Downcasting



Beneficios

Permite acceder a métodos y atributos específicos de la subclase.

Facilita el uso de objetos de la superclase como si como si fueran de la subclase, cuando es necesario.

Riesgos

ClassCastException: Si el objeto no es realmente una instancia de la subclase.

Requiere verificación con **instanceof** para evitar errores en tiempo de ejecución.

Ejemplo Práctico: Aplicando Polimorfismo y Arreglos con Casting



```
class Animal {
    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau!");
    }
    public void ladrar() {
        System.out.println("Ladrado fuerte");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Animal[] animales = new Animal[2];
        animales[0] = new Animal();
        animales[1] = new Perro();

        for (Animal animal : animales) {
            animal.hacerSonido();
            if (animal instanceof Perro) {
                Perro perro = (Perro) animal;
                perro.ladrar();
            }
        }
    }
}
```

Buenas Prácticas en el Uso de Polimorfismo y Casting



1 Usar interfaces

Define un contrato que las clases deben cumplir, promoviendo la flexibilidad.

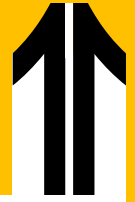
2 Evitar Downcasting innecesario

Reducir la necesidad de casting mejora la legibilidad y seguridad.

3 Priorizar Composición sobre Herencia

Ofrece más flexibilidad para reutilizar código y evitar problemas de herencia.

Errores Comunes y Cómo Evitarlos



ClassCastException

Ocurre al intentar downcastear un objeto a un tipo incorrecto. Usar **instanceof** antes del casting.

NullPointerException

Al acceder a un objeto nulo después de un casting fallido. Validar que el objeto no sea nulo.

Diseño inflexible

Abusar de la herencia y el casting puede llevar a un diseño rígido. Priorizar interfaces y composición. composición.

CONCLUSIONES Y PREGUNTAS

